

VIDEO 3 - BINARY TREES & BINARY SEARCH TREES

This script follows the BST presentation, including terminology, node representation, BST construction, duplicate handling, and all three traversals.

Opening / Title Slide

Hello everyone. In this final part of my BLA Lab 01, I will explain Binary Trees and Binary Search Trees, also called BSTs. I will discuss the difference between these two structures, important tree terminology, how a node can be represented in C++, how my BST is constructed, how duplicates are handled, and the three main traversal methods: Inorder, Preorder, and Postorder. I will then connect these ideas to the C++ implementation.

Binary Tree vs. Binary Search Tree

A Binary Tree is a tree data structure where each node can have at most two children, called the left child and right child. A node can have zero, one, or two children. A general Binary Tree does not require the values to be numerically ordered.

A Binary Search Tree is a special type of Binary Tree. In a BST, values smaller than the current node go into the left subtree, while larger values go into the right subtree. This ordering helps us decide which direction to follow when inserting or searching for a value.

Binary Tree Diagram

This diagram is an example of a general Binary Tree. The top value, 45, is the root. It has two children. Some other nodes also have children, while the nodes with no children are called leaf nodes.

The important point is that this is a Binary Tree, but the values do not have to follow the BST numerical ordering rule. That is why a larger value can appear on the left side of the root in a general Binary Tree.

BST Diagram and Ordering Rule

Now this diagram represents my Binary Search Tree. The root is 52. Because 31 is smaller than 52, it goes to the left. Because 74 is larger than 52, it goes to the right. We continue applying the same comparison rule at every node.

For example, 27 is smaller than 52, so we move left to 31. It is also smaller than 31, so we move left to 18. Then 27 is larger than 18, so it becomes the right child of 18. This shows that the BST rule is applied repeatedly as we move through the tree.

Tree Terminology

There are several important terms used with trees. Each value is stored in a node. The top node is called the root. A node that has another node below it is a parent, and the connected node below it is a child. A node with no children is called a leaf.

We can also divide a tree into left and right subtrees. A subtree contains a node and all nodes below it. In BST ordering, we can also discuss predecessor and successor. The predecessor is the next smaller value in sorted order, while the successor is the next larger value.

Basic Node Class in C++

After understanding the terminology, we can see how a single node is represented in C++. The Node class contains an integer variable called data, along with two pointers named left and right.

The data variable stores the actual value. The left pointer can connect the node to its left child or left subtree, while the right pointer connects it to the right side. By creating multiple Node objects and connecting these pointers, we can build the complete tree.

My Completed Binary Search Tree

I created this BST by inserting each unique value from my dataset in its original order. The first value, 52, becomes the root. Every later value is compared with the current node and moves left if it is smaller or right if it is larger.

The left subtree begins with 31 and the right subtree begins with 74. The leaf nodes are 27, 47, 59, 68, 79, and 92 because they have no children.

My dataset also contains duplicate values 31 and 74. In this implementation, duplicates are not inserted, so the repeated values are detected and ignored. For node 63, the predecessor is 59 and the successor is 68.

Inorder Traversal

The first traversal is Inorder. Its rule is Left, Root, Right. We first traverse the complete left subtree, then visit the current root, and then traverse the right subtree.

For my BST, the exact Inorder result is 18, 27, 31, 43, 47, 52, 59, 63, 68, 74, 79, 86, 92.

An important feature of a Binary Search Tree is that Inorder traversal produces the values in ascending sorted order because smaller values are stored on the left and larger values are stored on the right.

Preorder Traversal

Preorder follows the rule Root, Left, Right. This time, we visit the current root before traversing its children.

Starting with 52, we first record the root. Then we complete the left subtree beginning with 31, and after that we move to the right subtree beginning with 74.

The final Preorder result is 52, 31, 18, 27, 43, 47, 74, 63, 59, 68, 86, 79, 92.

Postorder Traversal

Postorder follows the rule Left, Right, Root. This means the parent is visited only after its left and right subtrees have been completed.

For my BST, the Postorder result is 27, 18, 47, 43, 31, 59, 68, 63, 79, 92, 86, 74, 52.

Notice that 52 is the main root and appears last. That happens because Postorder always processes the children before the parent.

Live C++ Demonstration

Now I will move to the C++ implementation. The program uses a Node class with data, left, and right members. The insert function works recursively. If the current position is empty, a new node is created. If the new value is smaller, the function continues into the left subtree. If it is larger, it continues into the right subtree. If the value is equal, the program identifies it as a duplicate and ignores it.

After building the tree, I call the Inorder, Preorder, and Postorder functions. Each function uses recursion but places the print statement in a different position. That small difference creates the three different traversal orders.

When I run the program, I will compare the output with the traversal results shown in my diagrams to confirm that the implementation matches the tree I constructed.

Closing

This completes my explanation of Binary Trees and Binary Search Trees. Through this part of the assignment, I learned how BST ordering determines node placement, how pointers connect nodes in C++, how duplicate values can be handled, and how recursive traversal algorithms visit the same tree in different orders.

I also want to give a special thanks to Professor Victor Govindaswamy. His teaching and the practice we do in class have helped me understand difficult Data Structure topics in a much easier way. The examples, quizzes, and programming work are helping me build my understanding step by step, especially for topics like Binary Trees and BST traversals.

Thank you for watching.